Lecture 4 — Advanced Tree Techniques

> World Finals Level · Prerequisites: *week-20* (tree DP), *week-22* (LCA, Euler tour), *nationals-02* (treaps / balanced BSTs), *nationals-04* (HLD, centroid decomposition, small-to-large) · Estimated time: 2 × 90 minutes

1. Overview

Nationals-04 covered the static tree toolkit: heavy-light decomposition for path queries, centroid decomposition for path counting, and small-to-large merging. This lecture adds four techniques that appear regularly in Codeforces Div1 E/F problems and at World Finals:

Technique	Solves	Cost
Link-cut tree	a forest that changes (link/cut/reroot) with path aggregates, online	O(log n) amortised per operation
DSU on tree ("sack")	one statistic per subtree, for all subtrees	O(n log n)
Virtual tree	many queries, each on a small set of marked vertices	O(k log n) per query
Rerooting DP	a tree DP for every choice of root	O(n)

Every reference program in Section 6 was stress-tested against a brute-force solution on random trees with shuffled labels.

2. Link-cut trees

2.1 Representation

A link-cut tree (Sleator and Tarjan, 1983) represents a forest of rooted trees. Every tree is partitioned into vertex-disjoint **preferred paths**, each going downward from some vertex. Each preferred path is stored in a **splay tree keyed by depth**: an in-order traversal lists the path from top to bottom. The root of each splay tree keeps a **path-parent** pointer to the vertex just above the top of its path, in the parent pointer field. A node's parent pointer is therefore either a real splay parent (the node is one of that parent's two children) or a path-parent (the node is *not* among the parent's children). `isRoot(x)` distinguishes the two cases.

2.2 access

`access(v)` restructures the preferred paths so that the path from the tree root to v is one preferred path ending at v:

1. Splay v and drop its right child (the part of the path deeper than v), which becomes a separate path whose path-parent is v.
2. While v's splay tree has a path-parent w: splay w, replace w's right child with v's splay tree (the old right child becomes a separate path), and continue from w.
3. Splay v so that it is the root of the single splay tree holding root…v.

Every other operation is access plus O(1) splay work:

- **makeRoot(v)**: access(v), then reverse the path (in-order order flips, so v becomes the shallowest). Reversal is a lazy flag that swaps children.
- **findRoot(v)**: access(v), then walk to the leftmost node of the splay (pushing lazies on the way), and splay that node.
- **link(u, v)**: makeRoot(u). If findRoot(v) ≠ u, set u's parent pointer to v (a path-parent).
- **cut(u, v)**: makeRoot(u), access(v). If the edge exists, the path is exactly u, v, so u is v's left child and has no right child. Detach it.
- **path aggregate(u, v)**: makeRoot(u), access(v), then read v's subtree aggregate in the splay.
- **LCA(u, v)** with a fixed root (no makeRoot calls): access(u), then access(v). The last vertex at which access(v) jumped via a path-parent is the LCA.

2.3 Complexity

The analysis has two parts.

- **Splay cost.** By the access lemma with a suitable weight function (the size of the subtree in the *represented* tree, including path-parent children), one access costs O(log n) amortised plus O(1) per *preferred-child change*.
- **Number of preferred-child changes.** Classify edges as heavy or light by subtree size in the represented tree. Each access makes at most $\log_2 n$ light edges preferred. A heavy edge can only become preferred after it stopped being preferred. That event is caused by an access that made a light edge preferred, or by a link/cut, so heavy changes are bounded by light changes plus O(log n) per link/cut.

Altogether m operations cost O((m + n) log n): **O(log n) amortised** per operation. The constant factor is larger than for segment trees. The reference implementation runs $3 \cdot 10^5$ mixed link/cut/query operations on $2 \cdot 10^5$ vertices in about 0.6 s, so plan on $2–5 \cdot 10^5$ operations per second.

2.4 Values on edges, other aggregates

LCT aggregates live on vertices. To put weights on edges, create one extra node per edge and link `u − e − v`, giving original vertices the neutral value. This is the standard approach for **dynamic MST**: when edge (u, v, w) arrives and u, v are already connected, query the maximum-weight edge node on the path u…v. If it is heavier than w, cut it and link the new edge.

Any associative aggregate works. A **non-commutative** aggregate (matrix products, "first and last element" summaries) must store both directions, because `makeRoot` reverses paths; swap the two directions in `flip`. Subtree aggregates (for example, the size of v's component when rooted at v) need "virtual subtree" values: each node also stores the total of its path-parent children, updated in `access` when the right child changes.

2.5 Where LCT appears

- Dynamic connectivity in a forest (SPOJ DYNACON1): link, cut, connected.
- Dynamic MST and "minimum bottleneck spanning tree while edges arrive".
- Offline problems where the alternative is a heavier technique. For example, maintaining a spanning forest that maximises deletion times gives a fully dynamic connectivity solution (an alternative to Lecture 5's segment tree over time).
- Path updates plus rerooting in the same problem, where HLD cannot follow the changing structure.

3. DSU on tree (sack)

3.1 The pattern

You need a statistic of each subtree (the most frequent colour, the number of distinct values at each depth, …). The statistic is easy to maintain in a global array under **add one vertex**, and the array can be **wiped completely**, but it does not support arbitrary removal.

The algorithm, for vertex v:

1. Recurse into every light child with `keep = false`. Each computes its answers and then wipes the global array.
2. Recurse into the heavy child with `keep = true`. Its contributions stay in the array.
3. Add every vertex of every light subtree, then v itself. The array now describes subtree(v), so answer v's queries.
4. If `keep = false`, remove every vertex of subtree(v) and reset any global summaries.

3.2 Complexity

A vertex u is added in step 3 once for each light edge on the path from the root to u (once per ancestor whose light child contains u), and once more as v itself. Every light edge at least halves the subtree size going down, so there are at most $\log_2 n$ of them. Removals mirror additions. Total: **O(n log n)** add/remove operations.

Use the Euler tour (`tin`, `tout`, `ver`) to iterate a light subtree as a contiguous range. It is simpler and faster than a recursive walk.

3.3 Non-removable statistics

In Codeforces 600E the statistic is "sum of colours with maximum count", and a max cannot be maintained under removal. The algorithm never removes partially, though: removal in step 4 always wipes the whole array, so resetting `maxCnt` and `sumMax` to zero is correct. This is the main advantage of sack over Mo's algorithm on the Euler tour.

3.4 Comparison

Small-to-large merging (*nationals-04*) also runs in O(n log n) merges, but with `std::set`/`map` containers the total is $O(n \log^2 n)$ and memory is less predictable. Sack uses one global array. Use small-to-large when the statistic is naturally a container per vertex (for example, a set of depths), and sack when it is a global counting array.

4. Virtual (auxiliary) trees

4.1 Problem shape

There are q queries. Each gives a set S of k marked vertices, with Σk bounded (say $2 \cdot 10^5$). An O(n) tree DP per query is too slow, but the DP only "branches" at marked vertices and at their LCAs. Unmarked vertices of degree two can be compressed into edges carrying aggregate information (length, minimum edge weight, ...).

4.2 Construction

Lemma. Sort S by DFS entry time: $a_1, \ldots, a_k$. The set $S \cup \{\mathrm{lca}(a_i, a_{i+1}) : 1 \le i < k\}$ is closed under LCA, and it is the vertex set of the virtual tree.

Proof sketch. Take two marked vertices a_i, a_j with i < j. In DFS order, every vertex between them lies in the subtree of `lca(a_i, a_j)`. The path from a_i to a_j in DFS order passes through the child subtrees of that LCA. Some adjacent pair `(a_t, a_{t+1})` with $i \le t < j$ straddles two

different child subtrees of it, and the LCA of that pair is exactly `lca(a_i, a_j)`. LCAs of LCAs reduce to the same argument.

So the virtual tree has at most 2k − 1 vertices. With the vertices sorted by `tin`, the virtual parent of `v_i` (for i ≥ 1) is `lca(v_{i−1}, v_i)`. The previous vertex in DFS order is either v's parent in the virtual tree or a descendant of a sibling branch, and their LCA is the lowest virtual ancestor of v in either case. A stack-based construction exists too, but this two-sort version is shorter and harder to get wrong.

Process vertices in **reverse** DFS order for bottom-up DP: every child comes after its parent in DFS order.

4.3 Example: 消耗战 (SDOI2011, Luogu P2495)

Given a weighted tree rooted at 1 and many queries of marked vertices, delete edges of minimum total weight so that no marked vertex is connected to the root. Precompute `mn[v]`, the minimum edge weight on the path root → v. On the virtual tree (with the root added):

```
cost(v) = mn[v]                          if v is marked (must cut above v)
cost(v) = min(mn[v], Σ_{children c} cost(c))   otherwise
answer  = Σ_{children c of root} cost(c)
```

Using `mn[v]` instead of the minimum over the compressed virtual edge alone is valid. Suppose the cheapest edge on root → v lies above v's virtual parent u. Then `mn[u]` equals that same weight, and u's option `min(mn[u], Σ …)` already pays for it once on behalf of all of u's subtree, so the DP never pays for a shared edge twice in an optimal choice. The brute-force comparison in testing confirms this on random trees.

Codeforces 613D *Kingdom and its Cities* uses the same construction with a different DP: count vertices to capture so that marked vertices are pairwise disconnected, or −1 if two marked vertices are adjacent.

4.4 Per-query discipline

- Reset only the vertices that appear in the virtual tree. Resetting arrays of size n per query is O(nq).
- Prefer an LCA with O(1) queries (Euler tour + sparse table) when Σk is very large. Binary lifting is fine for $\Sigma k \le 5 \cdot 10^5$.

5. Rerooting DP

5.1 Framework

We want `f(r)` for every root r, where `f` is a tree DP. Root the tree at 1.

1. **Down pass** (post-order): `down[v]` summarises subtree(v), computed from the children's `down` values.
2. **Up pass** (pre-order): `up[v]` summarises everything outside subtree(v), as seen from v through the edge to its parent. It is computed from `up[p]` and the `down` values of v's siblings.
3. **Answer**: `f(v) = merge(down[v], up[v])`.

5.2 Invertible merges

For sums (CSES 1133, sum of distances), moving the root from p to its child v brings `sz[v]` vertices one step closer and moves `n − sz[v]` vertices one step farther:

```
all[v] = all[p] − sz[v] + (n − sz[v]).
```

5.3 Non-invertible merges: prefix/suffix

For max/min, gcd, or any associative merge without an inverse, compute for each vertex p the prefix and suffix merges of its children's contributions. The contribution "everything at p except child c_i" is `merge(up[p], pre[i], suf[i+1])`. This costs O(deg p) per vertex, O(n) overall.

This also matters for modular products. "Dividing out" a child's factor fails when that factor is 0 mod p, a classic hidden bug, and prefix/suffix avoids it.

5.4 Iterative implementation

On a path graph with $n = 2 \cdot 10^5$, a recursive DFS goes $2 \cdot 10^5$ levels deep. Many judges allow this (Codeforces sets a large stack), but some do not. The reference code computes a preorder with an explicit stack, then runs the down pass over the reversed preorder and the up pass over the preorder. No recursion is needed.

6. Reference implementations

6.1 Link-cut tree (Library Checker *Dynamic Tree Vertex Add Path Sum*)

Input: `N Q`, the values `a_0..a_{N−1}`, N − 1 edges, then queries `0 u v w x` (cut u–v, link w–x), `1 p x` (add x to a_p) and `2 u v` (print the sum on the path u…v).

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// Link-cut tree over vertices 1..n: path sums, link, cut, connectivity, rerooting.
struct LinkCut {
    struct Node { int ch[2] = {0, 0}, p = 0; ll val = 0, sum = 0; bool rev = false; };
    vector<Node> t;                              // t[0] is the null sentinel
    LinkCut(int n) : t(n + 1) {}

    bool isRoot(int x) {                         // x is the root of its splay tree
        int p = t[x].p;
        return p == 0 || (t[p].ch[0] != x && t[p].ch[1] != x);
    }
    void pull(int x) { t[x].sum = t[t[x].ch[0]].sum + t[x].val + t[t[x].ch[1]].sum; }
    void flip(int x) { if (x) { swap(t[x].ch[0], t[x].ch[1]); t[x].rev ^= 1; } }
    void push(int x) { if (t[x].rev) { flip(t[x].ch[0]); flip(t[x].ch[1]); t[x].rev =
false; } }
    void rotate(int x) {
        int p = t[x].p, g = t[p].p, d = t[p].ch[1] == x;
        if (!isRoot(p)) t[g].ch[t[g].ch[1] == p] = x;   // test before relinking
        t[x].p = g;
        t[p].ch[d] = t[x].ch[d ^ 1];
        if (t[x].ch[d ^ 1]) t[t[x].ch[d ^ 1]].p = p;
        t[x].ch[d ^ 1] = p; t[p].p = x;
        pull(p); pull(x);
    }
    void splay(int x) {
        static vector<int> stk; stk.clear();
        for (int y = x; ; y = t[y].p) { stk.push_back(y); if (isRoot(y)) break; }
        while (!stk.empty()) push(stk.back()), stk.pop_back();   // push lazies top-down
        while (!isRoot(x)) {
            int p = t[x].p, g = t[p].p;
            if (!isRoot(p)) rotate((t[g].ch[1] == p) == (t[p].ch[1] == x) ? p : x);
            rotate(x);
        }
    }
    void access(int x) {                         // make root..x the preferred path
        for (int last = 0, y = x; y; last = y, y = t[y].p) {
            splay(y);
            t[y].ch[1] = last;
            pull(y);
        }
        splay(x);
    }
```

```
    void makeRoot(int x) { access(x); flip(x); }
    int findRoot(int x) {
        access(x);
        while (t[x].ch[0]) push(x), x = t[x].ch[0];
        splay(x);                                   // keeps the amortised bound
        return x;
    }
    bool connected(int u, int v) { return findRoot(u) == findRoot(v); }
    bool link(int u, int v) {                       // add edge u-v if they are in different
trees
        makeRoot(u);
        if (findRoot(v) == u) return false;
        t[u].p = v;
        return true;
    }
    bool cut(int u, int v) {                        // remove edge u-v if it exists
        makeRoot(u); access(v);
        if (t[v].ch[0] != u || t[u].ch[1] != 0) return false;
        t[v].ch[0] = t[u].p = 0;
        pull(v);
        return true;
    }
    void addVal(int x, ll d) { access(x); t[x].val += d; pull(x); }
    ll pathSum(int u, int v) { makeRoot(u); access(v); return t[v].sum; }
};

int main() {    // Library Checker: Dynamic Tree Vertex Add Path Sum
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    LinkCut lc(n);
    for (int i = 1; i <= n; i++) { ll a; cin >> a; lc.addVal(i, a); }
    for (int i = 0; i < n - 1; i++) { int u, v; cin >> u >> v; lc.link(u + 1, v + 1); }
    while (q--) {
        int type; cin >> type;
        if (type == 0) {
            int u, v, w, x; cin >> u >> v >> w >> x;
            lc.cut(u + 1, v + 1); lc.link(w + 1, x + 1);
        } else if (type == 1) {
            int p; ll x; cin >> p >> x; lc.addVal(p + 1, x);
        } else {
            int u, v; cin >> u >> v;
            cout << lc.pathSum(u + 1, v + 1) << "\n";
        }
    }
}
```

6.2 DSU on tree (Codeforces 600E *Lomsat gelral*)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```cpp
// Codeforces 600E Lomsat gelral: for every vertex, the sum of all colours that occur
// the maximum number of times in its subtree. DSU on tree, O(n log n).
int n, timer_ = 0;
vector<vector<int>> g;
vector<int> col, sz, heavy, tin, tout, ver, cnt;
vector<ll> ans;
int maxCnt = 0; ll sumMax = 0;

void prep(int v, int p) {
    sz[v] = 1; heavy[v] = 0; tin[v] = ++timer_; ver[timer_] = v;
    for (int c : g[v]) if (c != p) {
        prep(c, v);
        sz[v] += sz[c];
        if (!heavy[v] || sz[c] > sz[heavy[v]]) heavy[v] = c;
    }
    tout[v] = timer_;
}
void add(int v) {
    int c = col[v];
    if (++cnt[c] > maxCnt) maxCnt = cnt[c], sumMax = c;
    else if (cnt[c] == maxCnt) sumMax += c;
}
void dfs(int v, int p, bool keep) {
    for (int c : g[v]) if (c != p && c != heavy[v]) dfs(c, v, false);   // light
children, discard
    if (heavy[v]) dfs(heavy[v], v, true);                                // heavy child,
keep
    for (int c : g[v]) if (c != p && c != heavy[v])
        for (int t = tin[c]; t <= tout[c]; t++) add(ver[t]);             // re-add light
subtrees
    add(v);
    ans[v] = sumMax;
    if (!keep) {                                                         // wipe
everything
        for (int t = tin[v]; t <= tout[v]; t++) cnt[col[ver[t]]]--;
        maxCnt = 0; sumMax = 0;
    }
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    cin >> n;
    g.assign(n + 1, {}); col.assign(n + 1, 0); sz.assign(n + 1, 0); heavy.assign(n + 1,
0);
    tin.assign(n + 1, 0); tout.assign(n + 1, 0); ver.assign(n + 1, 0); cnt.assign(n + 1,
0);
    ans.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> col[i];
    for (int i = 0; i < n - 1; i++) {
        int x, y; cin >> x >> y;
        g[x].push_back(y); g[y].push_back(x);
    }
    prep(1, 0);
    dfs(1, 0, true);
```

```
    for (int i = 1; i <= n; i++) cout << ans[i] << " \n"[i == n];
}
```

6.3 Virtual tree ([SDOI2011] 消耗战, Luogu P2495)

Input: n, then n − 1 edges `u v w`, then m queries, each `k h_1 … h_k`. Output: the minimum cut cost per query.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// [SDOI2011] 消耗战 (Luogu P2495): for each query set H of vertices (1 not in H), find
the
// minimum total weight of edges to delete so that no vertex of H is connected to vertex
1.
// Virtual tree per query: O(k log n) per query.
int n, LOG;
vector<vector<pair<int,int>>> g;
vector<vector<int>> up;
vector<int> dep, tin;
vector<ll> mn;                                  // min edge weight on the path root..v

void build() {                                  // iterative DFS from vertex 1
    vector<int> st{1}; vector<char> vis(n + 1, 0);
    vis[1] = 1; up[0][1] = 1; mn[1] = LLONG_MAX; int timer = 0;
    while (!st.empty()) {
        int v = st.back(); st.pop_back(); tin[v] = timer++;
        for (auto [c, w] : g[v]) if (!vis[c]) {
            vis[c] = 1; up[0][c] = v; dep[c] = dep[v] + 1;
            mn[c] = min(mn[v], (ll)w); st.push_back(c);
        }
    }
    for (int k = 1; k < LOG; k++)
        for (int v = 1; v <= n; v++) up[k][v] = up[k - 1][up[k - 1][v]];
}
int lca(int a, int b) {
    if (dep[a] < dep[b]) swap(a, b);
    for (int k = LOG - 1; k >= 0; k--) if (dep[a] - (1 << k) >= dep[b]) a = up[k][a];
    if (a == b) return a;
    for (int k = LOG - 1; k >= 0; k--) if (up[k][a] != up[k][b]) a = up[k][a], b = up[k]
[b];
    return up[0][a];
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    cin >> n;
    LOG = 1; while ((1 << LOG) <= n) LOG++;
    g.assign(n + 1, {}); up.assign(LOG, vector<int>(n + 1)); dep.assign(n + 1, 0);
    tin.assign(n + 1, 0); mn.assign(n + 1, 0);
    for (int i = 0; i < n - 1; i++) {
```

```
        int u, v, w; cin >> u >> v >> w;
        g[u].push_back({v, w}); g[v].push_back({u, w});
    }
    build();
    vector<char> isKey(n + 1, 0);
    vector<ll> dp(n + 1, 0);
    vector<int> vpar(n + 1, 0);
    auto byTin = [&](int a, int b) { return tin[a] < tin[b]; };
    int m; cin >> m;
    while (m--) {
        int k; cin >> k;
        vector<int> vs(k);
        for (int& x : vs) { cin >> x; isKey[x] = 1; }
        vs.push_back(1);                          // root is always in the virtual tree
        sort(vs.begin(), vs.end(), byTin);
        int sz = vs.size();
        for (int i = 0; i + 1 < sz; i++) vs.push_back(lca(vs[i], vs[i + 1]));
        sort(vs.begin(), vs.end(), byTin);
        vs.erase(unique(vs.begin(), vs.end()), vs.end());
        for (size_t i = 1; i < vs.size(); i++) vpar[vs[i]] = lca(vs[i - 1], vs[i]);
        for (int v : vs) dp[v] = 0;
        for (int i = (int)vs.size() - 1; i > 0; i--) {   // reverse preorder: children
first
            int v = vs[i];
            ll cost = isKey[v] ? mn[v] : min(mn[v], dp[v]);
            dp[vpar[v]] += cost;
        }
        cout << dp[1] << "\n";
        for (int v : vs) isKey[v] = 0;           // reset only what we touched
    }
}
```

6.4 Rerooting with an invertible merge (CSES 1133 *Tree Distances II*)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// CSES 1133 Tree Distances II: for every vertex, the sum of distances to all vertices.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b;
        g[a].push_back(b); g[b].push_back(a);
    }
    vector<int> order, par(n + 1, 0);
    order.reserve(n);
    vector<int> st{1}; par[1] = -1;
    while (!st.empty()) {                        // iterative preorder (no recursion depth
issues)
        int v = st.back(); st.pop_back(); order.push_back(v);
```

```
        for (int c : g[v]) if (c != par[v]) par[c] = v, st.push_back(c);
    }
    vector<ll> sz(n + 1, 1), down(n + 1, 0), all(n + 1, 0);
    for (int i = n - 1; i > 0; i--) {           // bottom-up: down[v] = sum of depths in
subtree(v)
        int v = order[i], p = par[v];
        sz[p] += sz[v];
        down[p] += down[v] + sz[v];
    }
    all[1] = down[1];
    for (int i = 1; i < n; i++) {               // top-down: move the root across edge p-
v
        int v = order[i], p = par[v];
        all[v] = all[p] - sz[v] + (n - sz[v]);
    }
    for (int v = 1; v <= n; v++) cout << all[v] << " \n"[v == n];
}
```

6.5 Rerooting with prefix/suffix (CSES 1132 *Tree Distances I*)

```
#include <bits/stdc++.h>
using namespace std;

// CSES 1132 Tree Distances I: for every vertex, the maximum distance to any vertex.
// Rerooting with a non-invertible combine (max), using prefix/suffix maxima over
children.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b;
        g[a].push_back(b); g[b].push_back(a);
    }
    vector<int> order, par(n + 1, 0);
    vector<int> st{1}; par[1] = 0;
    while (!st.empty()) {
        int v = st.back(); st.pop_back(); order.push_back(v);
        for (int c : g[v]) if (c != par[v]) par[c] = v, st.push_back(c);
    }
    vector<int> down(n + 1, 0), up(n + 1, 0);   // up[v] = longest path starting at v
going via par[v]
    for (int i = n - 1; i >= 0; i--) {
        int v = order[i];
        for (int c : g[v]) if (c != par[v]) down[v] = max(down[v], down[c] + 1);
    }
    for (int v : order) {                       // parents before children
        vector<int> kids;
        for (int c : g[v]) if (c != par[v]) kids.push_back(c);
        int k = kids.size();
        vector<int> pre(k + 1, 0), suf(k + 1, 0);   // max of (down[c] + 1) over a prefix
/ suffix
        for (int i = 0; i < k; i++) pre[i + 1] = max(pre[i], down[kids[i]] + 1);
```

```
        for (int i = k - 1; i >= 0; i--) suf[i] = max(suf[i + 1], down[kids[i]] + 1);
        int fromAbove = (v == 1) ? 0 : up[v];
        for (int i = 0; i < k; i++)            // best path from v avoiding child i, plus edge v-c
            up[kids[i]] = 1 + max({fromAbove, pre[i], suf[i + 1]});
    }
    for (int v = 1; v <= n; v++) cout << max(down[v], v == 1 ? 0 : up[v]) << " \n"[v == n];
}
```

7. Common mistakes

- **LCT `rotate` order.** Test `isRoot(p)` before overwriting any pointers. Otherwise a path-parent edge is turned into a child edge and the structure silently corrupts.
- **LCT lazy reversal.** Push lazies from the top of the splay down to x *before* rotating. Push during the leftmost walk in `findRoot` as well.
- **LCT `cut` without verification.** If u and v are not adjacent, detaching v's left child cuts a different edge.
- **Sack global state.** Forgetting to reset summary variables (`maxCnt`, `sumMax`) after a wipe, or wiping only the light subtrees.
- **Virtual tree sorting by vertex id** instead of by DFS entry time. Or forgetting to include the root when the DP is defined relative to it.
- **Virtual tree resets.** Clearing whole arrays per query.
- **Rerooting double counting.** `up[c]` must exclude c's own contribution to p.
- **Deep recursion** on path-shaped trees. Use iterative traversals for anything that may be 10^5+ deep.
- **Unshuffled test generators.** Random trees generated as "parent of i is a random j < i" have labels that respect the rooting. Shuffle labels, or bugs that assume `par[v] < v` will hide.

8. Practice problems

1. Library Checker — *Dynamic Tree Vertex Add Path Sum* (LCT): https://judge.yosupo.jp/problem/dynamic_tree_vertex_add_path_sum
2. SPOJ DYNACON1 — *Dynamic Tree Connectivity*: https://www.spoj.com/problems/DYNACON1/
3. Luogu P3690 — LCT template (path xor, link, cut): https://www.luogu.com.cn/problem/P3690
4. Codeforces 600E — *Lomsat gelral*: https://codeforces.com/problemset/problem/600/E
5. Codeforces 570D — *Tree Requests* (sack by depth): https://codeforces.com/problemset/problem/570/D
6. Codeforces 208E — *Blood Cousins*: https://codeforces.com/problemset/problem/208/E
7. Codeforces 741D — *Arpa's letter-marked tree and Mehrdad's Dokhtar-kosh paths* (sack with xor masks): https://codeforces.com/problemset/problem/741/D

8. Codeforces 613D — *Kingdom and its Cities* (virtual tree): https://codeforces.com/problemset/problem/613/D
9. Luogu P2495 — [SDOI2011] 消耗战 (virtual tree): https://www.luogu.com.cn/problem/P2495
10. Codeforces 1320E — *Treeland and Viruses* (virtual tree + multi-source Dijkstra): https://codeforces.com/problemset/problem/1320/E
11. CSES 1132 / 1133 — *Tree Distances I / II*: https://cses.fi/problemset/task/1132, https://cses.fi/problemset/task/1133
12. Codeforces 1092F — *Tree with Maximum Cost* (rerooting): https://codeforces.com/problemset/problem/1092/F
13. Codeforces 1187E — *Tree Painting* (rerooting): https://codeforces.com/problemset/problem/1187/E

9. Further reading

- D. D. Sleator and R. E. Tarjan, "A data structure for dynamic trees", JCSS 26(3), 1983, and "Self-adjusting binary search trees", JACM 32(3), 1985.
- MIT 6.851 (Advanced Data Structures), lecture notes on dynamic graphs / link-cut trees: https://courses.csail.mit.edu/6.851/
- Codeforces blog by Arpa, "[Tutorial] Sack (dsu on tree)": https://codeforces.com/blog/entry/44351
- cp-algorithms, "Lowest Common Ancestor" (Euler tour + sparse table for O(1) LCA in virtual trees): https://cp-algorithms.com/graph/lca.html
- KACTL `LinkCutTree.h` for a compact notebook version: https://github.com/kth-competitive-programming/kactl